

WEEK – 4, 5

Introduction to Classes, objects, constructors, methods, parameter passing, overloading constructors and methods.

3. Classes and Objects CO1

Design a “farm animals” java application with the details of animals like cow, pig, horse.

Consider the following details like where they stay, what they eat, the sound they make by using classes and objects.

Steps For Execution:

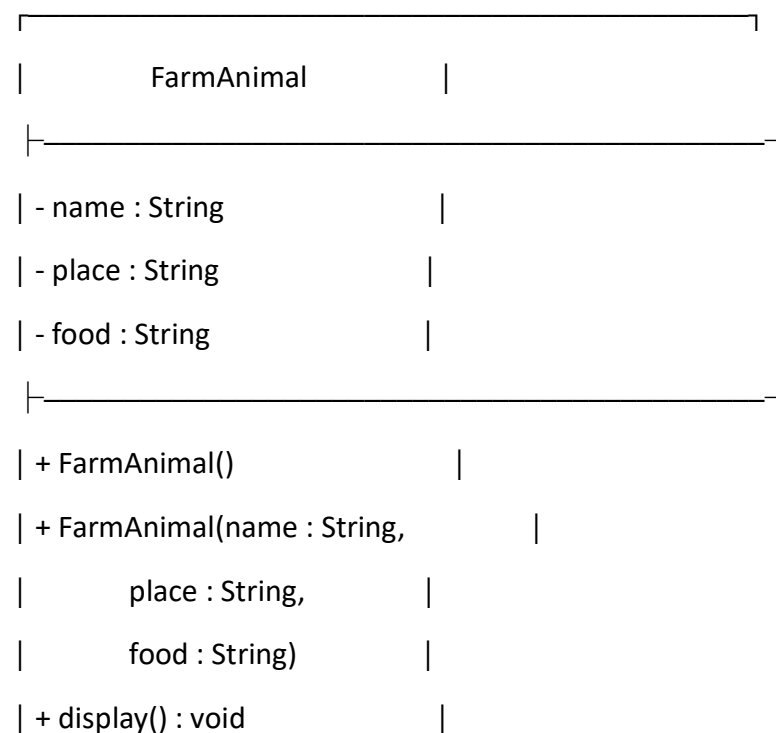
1. Create a folder named FarmAnimals.
2. Inside it, create a folder named record.
3. Save the Java program as FarmAnimals.java inside the record folder.
4. Open the FarmAnimals folder in IntelliJ IDEA/VS Code/Eclipse.
5. If there are no compilation errors, execute the program.
6. The program creates a FarmAnimal object for Cow using the parameterized constructor.
7. The program displays the Cow's name, place, and food.
8. The sound() method is called with "Moo" and the sound is displayed.
9. The program creates a FarmAnimal object for Pig using the parameterized constructor.
10. The program displays the Pig's name, place, and food.
11. The sound() method is called with "Oink" and the sound is displayed.
12. The program creates a FarmAnimal object for Horse using the parameterized constructor
13. The program displays the Horse's name, place, and food.
14. The overloaded sound() method is called with "Neigh" and 2.
15. The sound and number of times are displayed.
16. The program execution is completed.

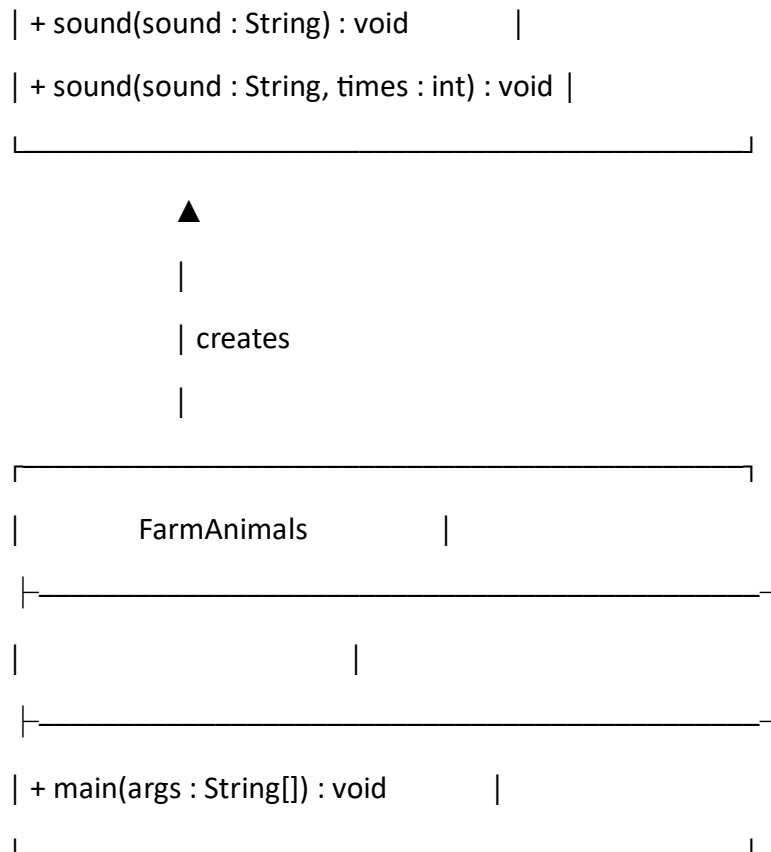
Algorithm:

1. Start.
2. Define the FarmAnimal class with data members:

- name, place, and food.
3. Define a default constructor to initialize default values.
 4. Define a parameterized constructor to initialize name, place, and food with given values.
 5. Define the display() method to display animal details.
 6. Define the sound(String sound) method to display the animal sound.
 7. Define the overloaded sound(String sound, int times) method to display the sound and number of times.
 8. In the main() method, create three FarmAnimal objects: Cow, Pig, and Horse using the parameterized constructor.
 9. Call display() and sound("Moo") for the Cow.
 10. Call display() and sound("Oink") for the Pig.
 11. Call display() and the overloaded sound("Neigh", 2) method for the Horse.
 12. Display all the animal details and sounds.
 13. Stop.

UML Class Diagram:





Code:

```
package javacore;
class FarmAnimal {

    // Data members
    String name;
    String place;
    String food;

    // Default constructor
    FarmAnimal() {
        name = "Unknown";
        place = "Farm";
        food = "Grass";
    }

    // Parameterized constructor
    FarmAnimal(String name, String place, String food) {
        this.name = name;
        this.place = place;
        this.food = food;
    }
}
```

```

// Method
void display() {
    System.out.println("Animal: " + name);
    System.out.println("Lives in: " + place);
    System.out.println("Eats: " + food);
}

// Method with parameter
void sound(String sound) {
    System.out.println("Sound: " + sound);
}

// Method overloading
void sound(String sound, int times) {
    System.out.println("Sound: " + sound);
    System.out.println("Number of times: " + times);
}
}

public class FarmAnimals {
    public static void main(String[] args) {

        // Objects using parameterized constructor
        FarmAnimal cow = new FarmAnimal(
            "Cow", "Cowshed", "Grass");

        FarmAnimal pig = new FarmAnimal(
            "Pig", "Pigsty", "Corn");

        FarmAnimal horse = new FarmAnimal(
            "Horse", "Stable", "Hay");

        // Display cow details
        cow.display();
        cow.sound("Moo");
        System.out.println();

        // Display pig details
        pig.display();
        pig.sound("Oink");
        System.out.println();

        // Display horse details
        horse.display();
        horse.sound("Neigh", 2);
    }
}

```

```
}  
}
```

Output:

Animal: Cow

Lives in: Cowshed

Eats: Grass

Sound: Moo

Animal: Pig

Lives in: Pigsty

Eats: Corn

Sound: Oink

Animal: Horse

Lives in: Stable

Eats: Hay

Sound: Neigh

Number of times: 2

4. Write a Java Program to below class diagram:

Steps For Execution:

1. Create a folder named DiagramBased.
2. Inside it, create a folder named record.
3. Save the Java program as DiagramBased.java inside the record folder.
4. Open the DiagramBased folder in IntelliJ IDEA/VS Code/Eclipse.
5. If there are no compilation errors, execute the program.
6. The program creates a Customer object and assigns:

Name = Reshma

Location = Visakhapatnam

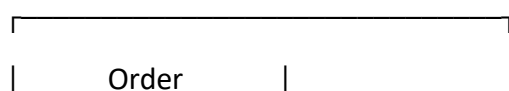
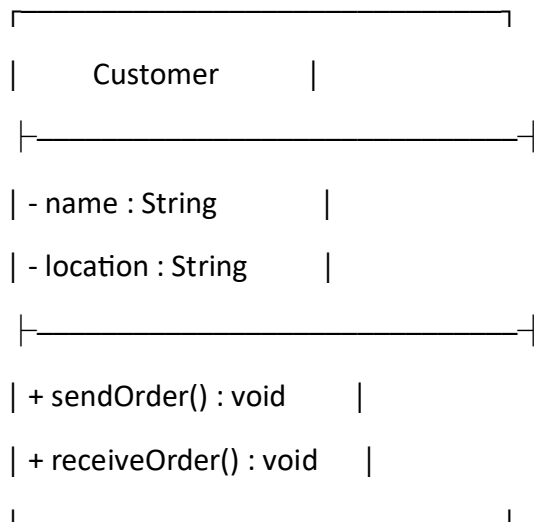
7. The sendOrder() method is called for the Customer.
The message "Reshma sent the order." is displayed.
8. The program creates a SpecialOrder object with:
Date = 13-08-2026
Number = S101
9. The confirm(), close(), and dispatch() methods are called
for the SpecialOrder.
10. The Special Order details and corresponding messages are displayed.
11. The program creates a NormalOrder object with:
Date = 13-08-2026
Number = N102
12. The confirm(), close(), dispatch(), and receive() methods
are called for the NormalOrder.
13. The Normal Order details and corresponding messages are displayed.
14. The receiveOrder() method is called for the Customer.
The message "Reshma received the order." is displayed.
15. The program execution is completed.

Algorithm:

1. Start.
2. Create the Customer class with name and location.
3. Define sendOrder() and receiveOrder() methods
to send and receive orders.
4. Create the Order class with date and number.
5. Define confirm() and close() methods for the Order.
6. Create SpecialOrder class that inherits from Order.
7. Define dispatch() method for SpecialOrder.
8. Create NormalOrder class that inherits from Order.
9. Define dispatch() and receive() methods for NormalOrder.

10. In the main() method, create a Customer object.
11. Assign the customer name as "Reshma" and location as "Visakhapatnam".
12. Call sendOrder() to display that the customer sent the order.
13. Create a SpecialOrder object and assign its date and order number.
14. Display the Special Order details.
15. Call confirm(), close(), and dispatch() methods for the Special Order.
16. Create a NormalOrder object and assign its date and order number.
17. Display the Normal Order details.
18. Call confirm(), close(), dispatch(), and receive() methods for the Normal Order.
19. Call receiveOrder() for the Customer.
20. Display all the required messages.
21. Stop.

UML Class Diagram:



|-----|

| - date : String |

| - number : String |

|-----|

| + confirm() : void |

| + close() : void |

|-----|

|

|-----|

|

|



|-----| |-----|

| SpecialOrder | | NormalOrder |

|-----| |-----|

| | |

|-----| |-----|

| + dispatch() | | + dispatch() |

| : void | | : void |

| | + receive() |

| | : void |

|-----| |-----|

|-----|

| DiagramBased |

|-----|

| |

|-----|

| + main(String[]) : void |

Code:

```
package javacore;
class Customer {
    String name;
    String location;

    void sendOrder() {
        System.out.println(name + " sent the order.");
    }

    void receiveOrder() {
        System.out.println(name + " received the order.");
    }
}

class Order {
    String date;
    String number;

    void confirm() {
        System.out.println("Order confirmed.");
    }

    void close() {
        System.out.println("Order closed.");
    }
}

class SpecialOrder extends Order {

    void dispatch() {
        System.out.println("Special order dispatched.");
    }
}

class NormalOrder extends Order {

    void dispatch() {
        System.out.println("Normal order dispatched.");
    }
}
```

```

void receive() {
    System.out.println("Normal order received.");
}
}

```

```

public class DiagramBased {
    public static void main(String[] args) {

        Customer c = new Customer();
        c.name = "Reshma";
        c.location = "Visakhapatnam";

        SpecialOrder so = new SpecialOrder();
        so.date = "13-08-2026";
        so.number = "S101";

        NormalOrder no = new NormalOrder();
        no.date = "13-08-2026";
        no.number = "N102";

        System.out.println("Customer: " + c.name);
        System.out.println("Location: " + c.location);
        c.sendOrder();

        System.out.println("\nSpecial Order");
        System.out.println("Date: " + so.date);
        System.out.println("Number: " + so.number);
        so.confirm();
        so.close();
        so.dispatch();

        System.out.println("\nNormal Order");
        System.out.println("Date: " + no.date);
        System.out.println("Number: " + no.number);
        no.confirm();
        no.close();
        no.dispatch();
        no.receive();

        c.receiveOrder();
    }
}

```

Output:

Customer: Reshma

Location: Visakhapatnam

Reshma sent the order.

Special Order

Date: 13-08-2026

Number: S101

Order confirmed.

Order closed.

Special order dispatched.

Normal Order

Date: 13-08-2026

Number: N102

Order confirmed.

Order closed.

Normal order dispatched.

Normal order received.

Reshma received the order.

5. Constructor overloading CO1

An organization is maintaining the data of employee according to cadre of employee with following parameters: name, id, designation, salary, promotion status. Apply the constructor overloading to implement it.

Steps For Execution:

1. Create a folder named EmployeeDetails.
2. Inside it, create a folder named record.

3. Save the Java program as EmployeeDetails.java inside the record folder.
4. Open the EmployeeDetails folder in IntelliJ IDEA/VS Code/Eclipse.
5. If there are no compilation errors, execute the program.
6. The program creates an Employee object e1 using the default constructor with:

Name = Ramesh

ID = 99

Designation = Manager

Salary = 50000

Promotion Status = Yes

7. The display() method is called for e1.

The details of Ramesh are displayed.

8. The program creates an Employee object e2 using the parameterized constructor with:

Name = Lalitha

ID = 101

Designation = Asst Manager

Salary = 40000

Promotion Status = Ye

9. The display() method is called for e2.

The details of Lalitha are displayed.

10. The program creates an Employee object e3 using the parameterized constructor with:

Name = Anil

ID = 102

Designation = Manager

Salary = 50000

Promotion Status = Promoted

11. The display() method is called for e3.

The details of Anil are displayed.

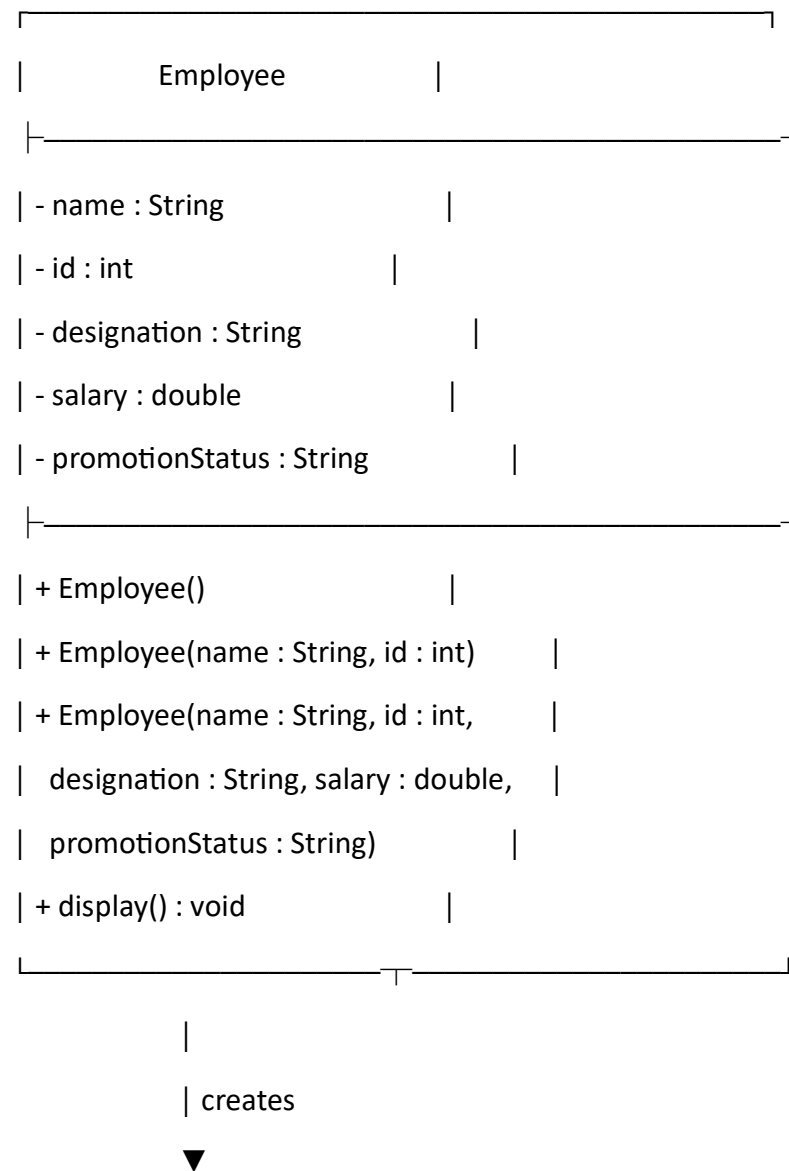
12. The program demonstrates constructor overloading by using three different constructors:
default constructor, two-parameter constructor, and five-parameter constructor.
13. The program execution is completed.

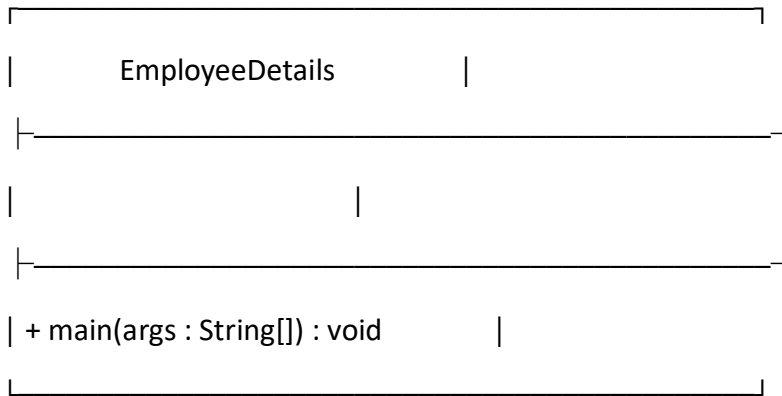
Algorithm:

1. Start.
2. Create the Employee class with the data members:
name, id, designation, salary, and promotionStatus.
3. Define a default constructor and initialize:
Name = Ramesh
ID = 99
Designation = Manager
Salary = 50000
Promotion Status = Yes
4. Define a parameterized constructor with name and id.
Initialize the employee details as:
Name = Lalitha
ID = 101
Designation = Asst Manager
Salary = 40000
Promotion Status = Yes
5. Define another parameterized constructor with:
name, id, designation, salary, and promotionStatus.
6. Define the display() method to display all employee details.
7. In the main() method, create employee object e1
using the default constructor.

8. Create employee object e2 using the two-parameter constructor.
9. Create employee object e3 using the five-parameter constructor with the given employee details.
10. Call display() for e1, e2, and e3.
11. Display the details of all three employees.
12. This demonstrates constructor overloading by using three constructors with different parameters.
13. Stop.

UML Class Diagram:





Code:

```
package javacore;
class Employee {
    String name;
    int id;
    String designation;
    double salary;
    String promotionStatus;

    Employee() {
        name = "Ramesh";
        id = 99;
        designation = "Manager";
        salary = 50000;
        promotionStatus = "Yes";
    }

    // Constructor 2: Name and ID
    Employee(String name, int id) {
        this.name = "Lalitha";
        this.id = 101;
        designation = "Asst Manager";
        salary = 40000;
        promotionStatus = "Yes";
    }

    Employee(String name, int id, String designation,
        double salary, String promotionStatus) {
        this.name = name;
```

```

        this.id = id;
        this.designation = designation;
        this.salary = salary;
        this.promotionStatus = promotionStatus;
    }

    void display() {
        System.out.println("Name: " + name);
        System.out.println("ID: " + id);
        System.out.println("Designation: " + designation);
        System.out.println("Salary: " + salary);
        System.out.println("Promotion Status: " + promotionStatus);
        System.out.println();
    }
}

public class EmployeeDetails {
    public static void main(String[] args) {

        Employee e1 = new Employee();

        Employee e2 = new Employee("Ravi", 101);

        Employee e3 = new Employee(
            "Anil", 102, "Manager", 50000, "Promoted"
        );

        e1.display();
        e2.display();
        e3.display();
    }
}

```

Output:

Name: Ramesh

ID: 99

Designation: Manager

Salary: 50000.0

Promotion Status: Yes

Name: Lalitha

ID: 101

Designation: Asst Manager

Salary: 40000.0

Promotion Status: Yes

Name: Anil

ID: 102

Designation: Manager

Salary: 50000.0

Promotion Status: Promoted